ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Examenes Anteriores del 2º Parcial SCD

Índice

1. Enunciado Carlos Ureña 2014-2015 **3**
1.1. Ejercicio 1. Corrección de errores (6 puntos) 3
1.2. Ejercicio 2. Problema de múltiples Productores-Consumidores con Paso de Mensajes (4 puntos) 3
1.3. Plantilla 4

2. Solución Carlos Ureña 2014-2015 **7**

1 Enunciado Carlos Ureña 2014-2015

Asignatura: Sistemas Concurrentes y Distribuidos.
Año Académico: 2014/2015.
Grado: Doble Grado en Ingeniería Informática y ADE + Matemáticas.
Descripción: Examen correspondiente a la práctica 3 de SCD.
Profesor: Carlos Ureña.
Fecha: 12 de enero de 2015.

1.1. Ejercicio 1. Corrección de errores (6 puntos)

Considerar el código del archivo disponible en la subsection de plantilla de este documento o aquí, que intenta plantear una solución al problema de la cena de los filósofos con camarero. Hay diversos errores en el código proporcionado que deberás solucionar:

1. Intenta compilar el código proporcionado. Comprueba que hay diversos errores básicos de sintaxis que impiden que el código genere un ejecutable. Encuentra dichos errores y arréglalos hasta que el código compile correctamente. Indica en el folio del examen el número de línea y cómo debería quedar ésta una vez solventado el problema.

2. Ejecuta el programa y observa la salida que genera. El algoritmo presenta un error en su diseño. Explica brevemente en el folio del examen por qué la salida proporcionada no es correcta.

3. Observa el código del programa ejecutado. Encuentra y corrige el error en el diseño del algoritmo que impide que el programa muestre una salida coherente. Señala en el folio del examen en qué línea está el problema y cómo debería quedar ésta para solucionar el error.

4. Se desea modificar el algoritmo dado del problema de los filósofos con camarero para dar servicio a 7 filósofos y 7 tenedores en lugar de a 5. También se desea cambiar el identificador de los procesos para que los filósofos sean los procesos con rank 0, 1, 2, ..., 6 y los tenedores sean los procesos 7, 8, ..., 13. Describe brevemente qué cambios debes realizar en el código proporcionado y en qué líneas.

1.2. Ejercicio 2. Problema de múltiples Productores-Consumidores con Paso de Mensajes (4 puntos)

Partiendo del algoritmo desarrollado durante la Práctica 3 para resolver el problema de múltiples Productores-Consumidores usando paso de mensajes MPI, se pide realizar los siguientes cambios:

- El valor que los procesos productores que envían al buffer debe ser un valor aleatorio entre 0 y 9 en lugar de uno secuencial como hasta ahora.

- Así mismo, el productor deberá enviar un mensaje al buffer pidiendo permiso para enviar el valor, y el buffer deberá responder al productor autorizando a enviar el mensaje antes de hacerlo (al igual que ya hace el consumidor).

Realiza los cambios necesarios para que, en el caso de que en el buffer se almacenen dos valores iguales consecutivos (independientemente del productor que los envíe), el buffer finalice.

- Antes de finalizar deberá avisar a todos los productores y consumidores para que estos finalicen enviando un valor especial en los mensajes de autorización (por ejemplo, un valor negativo). Sólo cuando todos los procesos hayan finalizado el buffer terminará.

- Implementa esta condición de forma que cuando cualquier proceso vaya a finalizar, imprima un mensaje en pantalla informando de ello.

1.3. Plantilla

```
#include <iostream>
#include <time.h>
#include <unistd.h>
#include <stdlib.h>
#include <mpi.h>

#define NUM_FILOSOFOS 5
#define NUM_PROCESOS (2*(NUM_FILOSOFOS)+1)
#define RANK_CAMARERO ((NUM_PROCESOS)-1)

#define TAG_SENTARSE 0
#define TAG_LEVANTARSE 1

using namespace std;
void Filosofo(int id);
void Tenedor(int id);
void Camarero();

// ----------------------------------------------------------------------

void retraso_aleatorio()
{
    static bool primera = true;

    // inicializar generador de números aleatorios (la primera vez)
    if (primera)
    {
        srand(time(NULL));
        primera = false;
    }
    // retraso aleatorio, de entre una y dos décimas de segundo
    usleep(1000L * (100L + rand() % 100L));
}

// ----------------------------------------------------------------------
```

```

int main(int argc, char **argv)
{
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != NUM_PROCESOS)
    {
        if (rank == 0)
            cout << "El numero de procesos debe ser " << NUM_PROCESOS << endl << flush;
        MPI_Finalize();
        return 0;
    }

    if (rank == RANK_CAMARERO)
        Camarero(); // el 10 es el camarero
    else if ((rank % 2) == 0)
        Filosofo(rank); // los pares (0,2,4,6,8) son Filosofos
    else
        Tenedor(rank); // los impares (1,3,5,7,9) son Tenedores

    MPI_Finalize();
    return 0;
}

// ---------------------------------------------------------------------

void Filosofo(int id)
{
    const int
        primero = (id + 1) % (NUM_PROCESOS - 1),
        segundo = (id + (NUM_PROCESOS - 2)) % (NUM_PROCESOS - 1);

    while (true)
    {
        // solicita sentarse
        cout << "Filosofo " << id << " solicita sentarse." << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, RANK_CAMARERO, TAG_SENTARSE, MPI_COMM_WORLD);

        // solicita y coge primer tenedor
        cout << "Filosofo " << id << " solicita primer tenedor: " << primero << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD);
        cout << "Filosofo " << id << " coge primer tenedor: " << primero << endl << flush;

        // solicita coge segundo tenedor
        cout << "Filosofo " << id << " coge segundo tenedor: " << segundo << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, segundo, MPI_COMM_WORLD);
```

```
        cout << "Filosofo " << id << " coge segundo tenedor: " << segundo << endl << flush;

        // come
        cout << "Filosofo " << id << " COMIENDO" << endl << flush;
        retraso_aleatorio();

        // suelta el segundo tenedor
        cout << "Filosofo " << id << " suelta tenedor: " << segundo << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD);

        // suelta el primer tenedor
        cout << "Filosofo " << id << " suelta tenedor: " << primero << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD);

        // solicita levantarse
        cout << "Filosofo " << id << " solicita levantarse." << endl << flush;
        MPI_Ssend(&id, 1, MPI_INT, RANK_CAMARERO, TAG_LEVANTARSE, MPI_COMM_WORLD);

        // piensa
        cout << "Filosofo " << id << " PENSANDO" << endl << flush;
        retraso_aleatorio();
    }
}

// ----------------------------------------------------------------------

void Tenedor(int id)
{
    int valor;
    MPI_Status status;

    while (true)
    {
        // espera un mensaje desde cualquier filosofo vecino
        cout << "Tenedor " << id << " espera petición." << endl << flush;
        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, id, MPI_COMM_WORLD, &status);
        cout << "Tenedor " << id << " recibe petición de " << status.MPI_SOURCE << endl << flush;

        // espera a que el filosofo suelte el tenedor
        MPI_Recv(&valor, 1, MPI_INT, status.MPI_SOURCE, id, MPI_COMM_WORLD, &status);
        cout << "Tenedor " << id << " recibe liberación de " << status.MPI_SOURCE << endl << flush;
    }
}

// ----------------------------------------------------------------------

void Camarero()
```

```
133 {
134     int valor, num_sentados = 0, tag_aceptada;
135     MPI_Status status;
136 
137     while (true)
138     {
139         if (num_sentados < NUM_FILOSOFOS - 1)
140             tag_aceptada = MPI_ANY_TAG;
141         else
142             tag_aceptada = TAG_LEVANTARSE;
143 
144         // espera un mensaje desde cualquier filosofo con etiqueta
            aceptable
145         cout << "Camarero espera petición de sentarse/levantarse (sentados
            == " << num_sentados << ")" << endl << flush;
146 
147         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, tag_aceptada,
            MPI_COMM_WORLD, &status);
148         cout << "Camarero recibe petición de filosofo " << status.
            MPI_SOURCE << endl << flush;
149 
150         if (status.MPI_TAG == TAG_LEVANTARSE)
151             num_sentados--;
152         else
153             num_sentados++;
154     }
155 }
```

La plantilla se encuentra aquí.

2 Solución Carlos Ureña 2014-2015

Ejercicio 1

- Errores:
  - En la línea 84 falta un argumento en la función `MPI_Ssend`. No está puesto el tag del mensaje (debería ser "segundo").
  - En la línea 124 que los dos primeros argumentos están al revés.
- Error en el diseño: podemos ver que tras la ejecución del programa, los filósofos no se levantan de la mesa. El error se encuentra en la línea 101, en el paso del mensaje al camarero para solicitar levantarse, el tag del mensaje es el de sentarse (`TAG_SENTARSE`), en lugar del de levantarse (`TAG_LEVANTARSE`).
- Último apartado:
  - para dar servicio a 7 filósofos y 7 tenedores, se deben cambiar las constantes `NUM_FILOSOFOS` en la línea 7.
  - para ello debemos de cambiar la línea 54 poniendo que el rank debe de ser menor al número de filósofos.

- además, debemos de cambiar la asignación de los tenedores en la línea 68 y 69, debido a que la presente no es coherente ni consistente desembocando en un error en el que varios filósofos quieren coger el mismo tenedor.

Ejercicio 2

En la resolución de este ejercicio usamos esta plantilla.

- Hemos añadido en la línea 58 en la función de producir valor esta línea **int valor = aleatorio<0, 9>();** para que produzca el valor aleatorio.
- En la función del productor debemos añadir **MPI_Ssend(&peticion, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD);** para que el productor pida permiso para enviar el valor y añadimos en el **switch** que cuando sea el caso del productor reciba la petición de aviso: **MPI_Recv(&peticion_productor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_productor, MPI_COMM_WORLD, &estado);**

Para la solución del apartado 2.2 revisa este código donde los comentarios **apartado2.2** reflejan los cambios que se han realizado.

```
//
    ------------------------------------------------------------------------
//
// Sistemas concurrentes y Distribuidos.
// Práctica 3. Implementación de algoritmos distribuidos con MPI
//
// Archivo: prodcons2.cpp
// Implementación del problema del productor-consumidor con
// un proceso intermedio que gestiona un buffer finito y recibe peticiones
// en orden arbitrario
// (versión con un único productor y un único consumidor)
//
// Historial:
// Actualizado a C++11 en Septiembre de 2017
//
    ------------------------------------------------------------------------

#include <iostream>
#include <thread> // this_thread::sleep_for
#include <random> // dispositivos, generadores y distribuciones aleatorias
#include <chrono> // duraciones (duration), unidades de tiempo
#include <mpi.h>

using namespace std;
using namespace std::this_thread ;
using namespace std::chrono ;

const int
   num_items              = 20,
   tam_vector             = 10,
   n_p = 4,
   n_c = 5,
```

```
   num_procesos_esperado = n_p + n_c + 1,
   k = num_items/n_p,
   valor_especial        = -1; // apartado 2.2: valor especial para finalizar

//variables para el test de produccion de valores
int valores_producidos[n_p] = {0};
int valores_consumidos [n_c] = {0};

const int etiq_productor = 0, // etiqueta de productor
    etiq_consumidor = 1; // etiqueta de consumidor


//*********************************************************************
// plantilla de función para generar un entero aleatorio uniformemente
// distribuido entre dos valores enteros, ambos incluidos
// (ambos tienen que ser dos constantes, conocidas en tiempo de compilación)
//---------------------------------------------------------------------

template< int min, int max > int aleatorio()
{
  static default_random_engine generador( (random_device())() );
  static uniform_int_distribution<int> distribucion_uniforme( min, max ) ;
  return distribucion_uniforme( generador );
}
// ---------------------------------------------------------------------
// producir produce un número aleatorio entre 0 y 9
// y lleva espera aleatorio
int producir(int numero_productor) {
   int valor = aleatorio<0, 9>();
   sleep_for(milliseconds(aleatorio<10, 100>()));
   cout << "Productor " << numero_productor << " ha producido valor " << valor << endl << flush;
   valores_producidos[numero_productor]++;
   return valor;
}
// ---------------------------------------------------------------------

void funcion_productor(int numero_productor) {
   int peticion = 1;
   for (int i = 0; i < k; i++) {
      int valor_prod = producir(numero_productor);
      MPI_Ssend(&peticion, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD);
      MPI_Ssend(&valor_prod, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD);
      cout << "Productor " << numero_productor << " envió valor " << valor_prod << endl << flush;
   }
   // apartado 2.2: enviar valor especial para finalizar
   MPI_Ssend(&peticion, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD);
   MPI_Ssend(&valor_especial, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD);
   cout << "Productor " << numero_productor << " finaliza." << endl << flush;
```

```
}
// ----------------------------------------------------------------------

void consumir( int valor_cons )
{
   // espera bloqueada
   sleep_for( milliseconds( aleatorio<110,200>()) );
   cout << "Consumidor ha consumido valor " << valor_cons << endl << flush ;
}
// ----------------------------------------------------------------------

void funcion_consumidor(int numero_consumidor)
{
   int         peticion = 1,
               valor_rec;
   MPI_Status  estado ;

   while (true) // apartado 2.2: cambiar a bucle infinito
   {
      MPI_Ssend( &peticion,  1, MPI_INT, n_p, etiq_consumidor, MPI_COMM_WORLD); //n_p hace referencia al id de buffer
      MPI_Recv ( &valor_rec, 1, MPI_INT, n_p, etiq_consumidor, MPI_COMM_WORLD,&estado );
      if (valor_rec == valor_especial) { // apartado 2.2: verificar valor especial
         cout << "Consumidor " << numero_consumidor << " finaliza." << endl << flush;
         break;
      }
      cout << "Consumidor con rol " << numero_consumidor << " ha recibido valor " << valor_rec << endl << flush ;
      consumir( valor_rec );
      valores_consumidos[numero_consumidor]++;
   }
}
// ----------------------------------------------------------------------

void funcion_buffer()
{
   int        buffer[tam_vector],      // buffer con celdas ocupadas y vacías
              valor,                   // valor recibido o enviado
              primera_libre       = 0, // índice de primera celda libre
              primera_ocupada     = 0, // índice de primera celda ocupada
              num_celdas_ocupadas = 0, // número de celdas ocupadas
              ultimo_valor        = -2; // apartado 2.2: último valor recibido
   MPI_Status estado ;                 // metadatos del mensaje recibido

   int etiq_aceptable; // etiqueta aceptable, almacena el valor de la etiqueta del mensaje recibido
   int peticion_productor = 0;
   bool finalizar = false; // apartado 2.2: bandera para finalizar

   while (!finalizar) // apartado 2.2: cambiar a bucle infinito
```

```
    {
        etiq_aceptable = (num_celdas_ocupadas == 0) ? etiq_productor
                         : (num_celdas_ocupadas == tam_vector) ? etiq_consumidor
                         : MPI_ANY_TAG;

        // 2. recibir un mensaje del emisor o emisores aceptables

        MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_aceptable, MPI_COMM_WORLD, &estado );

        // 3. procesar el mensaje recibido

        switch( estado.MPI_TAG ) // leer emisor del mensaje en metadatos
        {
            case etiq_productor: // si ha sido el productor: insertar en buffer
                MPI_Recv( &peticion_productor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_productor, MPI_COMM_WORLD, &estado );
                if (valor == valor_especial) { // apartado 2.2: verificar valor especial
                    finalizar = true;
                    break;
                }
                buffer[primera_libre] = valor ;
                primera_libre = (primera_libre+1) % tam_vector ;
                num_celdas_ocupadas++ ;
                cout << "Buffer ha recibido valor " << valor << endl ;
                if (valor == ultimo_valor) { // apartado 2.2: verificar valores consecutivos
                    finalizar = true;
                }
                ultimo_valor = valor;
                break;

            case etiq_consumidor: // si ha sido el consumidor: extraer y enviarle
                valor = buffer[primera_ocupada] ;
                primera_ocupada = (primera_ocupada+1) % tam_vector ;
                num_celdas_ocupadas-- ;
                cout << "Buffer va a enviar valor " << valor << endl ;
                MPI_Ssend( &valor, 1, MPI_INT, estado.MPI_SOURCE , etiq_consumidor, MPI_COMM_WORLD);
                break;
        }
    }

    // apartado 2.2: notificar a todos los productores y consumidores para finalizar
    for (int i = 0; i < n_p; i++) {
        MPI_Ssend(&peticion_productor, 1, MPI_INT, i, etiq_productor, MPI_COMM_WORLD);
        MPI_Ssend(&valor_especial, 1, MPI_INT, i, etiq_productor, MPI_COMM_WORLD);
    }
    for (int i = 0; i < n_c; i++) {
```

```
        MPI_Ssend(&peticion_productor, 1, MPI_INT, n_p + 1 + i, etiq_consumidor, MPI_COMM_WORLD);
        MPI_Ssend(&valor_especial, 1, MPI_INT, n_p + 1 + i, etiq_consumidor, MPI_COMM_WORLD);
    }
    cout << "Buffer finaliza." << endl << flush;
}

//Función extra para verificar que cada productor ha producido la cantidad correcta de ítems
void test_produccion() {

    for (int i = 0; i < n_p; i++)
    {
        if(valores_producidos[i] == k){
            cout << endl << "------------------------------------------------------------" << endl;
            cout << "| Productor " << i << " ha producido la cantidad correcta de ítems |" << endl;
            cout << "------------------------------------------------------------" << endl;
        }
    }

    for (int i = 0; i < n_c; i++)
    {
        if (valores_consumidos[i] == num_items/n_c)
        {
            cout << endl << "------------------------------------------------------------" << endl;
            cout << "| Consumidor " << i << " ha consumido la cantidad correcta de ítems |" << endl;
            cout << "------------------------------------------------------------" << endl;
        }
    }




}


// ----------------------------------------------------------------------
int main( int argc, char *argv[] )
{
    int id_propio, num_procesos_actual;

    // inicializar MPI, leer identif. de proceso y número de procesos
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
    MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );
```

```
    if(num_procesos_esperado == num_procesos_actual) {
        if(id_propio < n_p) {
            funcion_productor(id_propio);

        }
        else if (id_propio == n_p) {
            funcion_buffer();

        }
        else {
            funcion_consumidor(id_propio - n_p - 1); // debemos de restarlo para que coincida con el numero del proceso consumidor
        }
    } else {
        if ( id_propio == 0 ) // solo el primero escribe error, indep. del rol
        { cout << "el número de procesos esperados es:    " << num_procesos_esperado << endl
            << "el número de procesos en ejecución es: " << num_procesos_actual << endl
            << "(programa abortado)" << endl ;
        }
    }

    // al terminar el proceso, finalizar MPI
    MPI_Finalize( );

    test_produccion();
    return 0;
}
```